

Paterni ponašanja

1. Strategy – izdvaja algoritam iz matične klase i uključuje ga u posebne klase.

U našem projektu, Strategy pattern može se koristiti za implementaciju različitih algoritama preporuka i rangiranja filmova i serija. Sistem BMDb omogućava personalizovane preporuke na osnovu omiljenih žanrova korisnika, kao i globalno rangiranje najpopularnijeg sadržaja. Umjesto korištenja velikog broja if-else grananja, uvode se posebne strategije poput preporuke po žanru, preporuke na osnovu ocjena ili preporuke na osnovu popularnosti. Time se omogućava jednostavna zamjena algoritama tokom izvršavanja programa bez izmjene ostatka sistema. Ovakav pristup omogućava proširenje sistema novim algoritmima bez narušavanja Open/Closed principa.

2. State – mijenja način ponašanja objekata na osnovu trenutnog stanja.

State pattern može se primijeniti nad korisnicima i oglasima u BMDb sistemu. Korisnik može imati stanja poput aktivan, blokiran ili verifikovan recenzent, dok oglas može biti aktivan ili deaktiviran. Umjesto korištenja switch ili if logike za provjeru stanja korisnika ili oglasa, svako stanje bi bilo implementirano u posebnoj klasi. Na taj način objekti mijenjaju svoje ponašanje automatski u zavisnosti od trenutnog stanja. Ovim pristupom postiže se bolja organizacija koda, lakše održavanje sistema i uklanjanje kompleksnih uslovnih grananja.

3. Template Method – omogućava algoritmima da izdvoje pojedine korake u podklase.

Template Method pattern može se koristiti za procese objavljivanja sadržaja u sistemu. Na primjer, dodavanje filma, serije ili recenzije uvijek prati isti tok: validacija podataka, spremanje u bazu i slanje notifikacije korisnicima. Baza klasa definiše zajednički kostur algoritma, dok konkretne klase implementiraju specifične detalje za filmove, serije ili recenzije. Time se izbjegava dupliranje koda i osigurava konzistentan tok izvršavanja operacija kroz cijeli sistem. Ovaj pristup olakšava buduće proširenje funkcionalnosti bez mijenjanja osnovne strukture algoritma.

4. Observer – uspostavlja relaciju između objekata takvu da kad se stanje jednog objekta promijeni svi vezani objekti dobiju informaciju.

Observer pattern se može koristiti za implementaciju sistema notifikacija u BMDb aplikaciji. Kada se doda novi film, nova serija ili se promijeni status sadržaja, sistem automatski obavještava registrovane korisnike koji su pretplaćeni na notifikacije. Klasa koja predstavlja sadržaj ima ulogu Subject objekta, dok korisnici predstavljaju Observer objekte. Kada se stanje sadržaja promijeni, svi registrovani korisnici dobijaju obavijest bez potrebe za direktnim povezivanjem klasa. Ovakav pristup omogućava labavo povezivanje sistema i jednostavno dodavanje novih vrsta notifikacija.

5. Iterator – omogućava pristup elementima kolekcije sekvencijalno bez poznavanja interne strukture kolekcije.

Iterator pattern može se koristiti za prolazak kroz kolekcije podataka u BMDb sistemu. Sistem sadrži veliki broj lista kao što su lista filmova, lista serija, lista recenzija, lista glumaca, lista notifikacija i korisničke liste „Gledao sam“ i „Gledat ću“. Iterator omogućava jednostavan prolazak kroz te kolekcije bez potrebe da klijent zna na koji način su podaci interno organizovani. Time se poboljšava čitljivost i održavanje koda, jer različite kolekcije mogu koristiti isti način pristupa podacima. Također omogućava implementaciju različitih načina prolaska kroz sadržaj bez izmjene postojećih klasa.

6. Command – pretvara zahtjev u samostalni objekt, omogućavajući parametrizaciju klijenata različitim zahtjevima.

Command pattern može se koristiti za operacije poput dodavanja recenzije, brisanja recenzije, blokiranja korisnika ili aktivacije oglasa. Svaka akcija korisnika predstavlja poseban Command objekat koji sadrži potrebne informacije za izvršavanje operacije. Na primjer, klik na dugme „Objavi recenziju“ kreira objekat PublishReviewCommand koji izvršava logiku objavljivanja. Ovakav pristup odvaja korisnički interfejs od poslovne logike sistema i omogućava jednostavno logovanje akcija, undo/redo funkcionalnosti i lakše testiranje sistema.

7. Mediator – enkapsulira protokol za komunikaciju među objektima dozvoljavajući da objekti komuniciraju bez međusobnog poznavanja interne strukture.

Mediator pattern može se koristiti za koordinaciju komunikacije između modula notifikacija, oglasa, recenzija i korisnika. Umjesto da klase direktno komuniciraju jedna s drugom, uvodi se centralni mediator koji upravlja razmjenom informacija. Na primjer, kada moderator obriše neprimjerenu recenziju, mediator automatski obavještava sistem notifikacija i ažurira statistiku sadržaja bez direktnog povezivanja tih klasa. Time se značajno smanjuje zavisnost između objekata i poboljšava modularnost sistema. Ovakav pristup čini kod preglednijim i jednostavnijim za održavanje.

8. Chain of Responsibility – predstavlja listu objekata, ukoliko objekat ne može da odgovori prosljeđuje zahtjev narednom u nizu.

Chain of Responsibility pattern može se koristiti prilikom obrade zahtjeva za dodavanje sadržaja u sistem. Zahtjev prolazi kroz više koraka obrade kao što su validacija podataka, provjera dozvoljenih formata fajlova, spremanje sadržaja i slanje notifikacija. Svaki objekat u lancu obrađuje svoj dio zahtjeva ili ga prosljeđuje sljedećem objektu. Na taj način moguće je jednostavno dodati nove korake obrade bez izmjene postojećeg toka sistema. Ovaj pristup povećava fleksibilnost i modularnost aplikacije.

9. Visitor – definiše i izvršava nove operacije nad elementima postojeće strukture bez mijenjanja same strukture.

Visitor pattern može se koristiti za izvršavanje dodatnih operacija nad objektima kao što su Film, Serija, Recenzija ili Korisnik. Na primjer, moguće je implementirati generisanje statističkih izvještaja, izračun popularnosti sadržaja ili eksport podataka bez izmjene postojećih klasa modela. Novi tipovi operacija dodaju se kroz posebne Visitor klase, dok struktura postojećih objekata ostaje nepromijenjena. Time se postiže lakše proširenje funkcionalnosti sistema i poštuje Open/Closed princip.

10. Memento – omogućava spašavanje internog stanja nekog objekta van sistema i njegovo ponovno vraćanje.

Memento pattern može se koristiti za vraćanje prethodnih verzija recenzija ili korisničkih podataka. Kada korisnik izmijeni recenziju ili moderator promijeni podatke o filmu, sistem može sačuvati prethodno stanje objekta. Ukoliko korisnik želi vratiti staru verziju ili dođe do greške, moguće je obnoviti prethodno stanje bez uticaja na ostatak sistema. Ovaj pattern omogućava implementaciju undo funkcionalnosti i povećava sigurnost rada sa podacima.